

OBJECT AND CLASSES**OBJECT**

- ❖ Everything in the world is an object.
- ❖ Object is physical representation of class.
- ❖ Object is the main building block of OOPS i.e. our applications will be divided into multiple objects.
- ❖ Object is the run-time entity because memory will be allocated for an object while executing the program as per class description.
- ❖ It is also called as instance of the class so both object and instance are interchangeable.

Syntax:

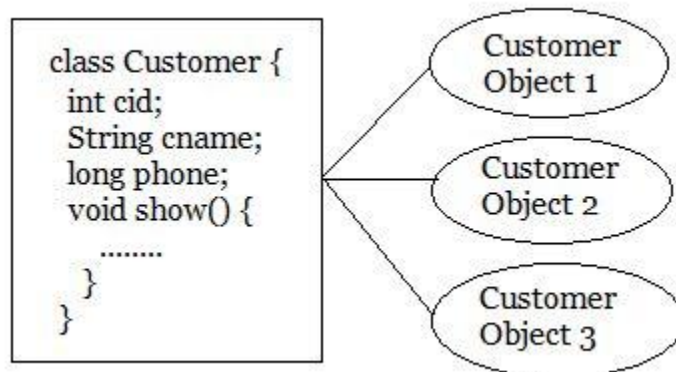
```
<ClassName><refVarName>=new <ClassName>();
```

Example:

```
Customer c1=new Customer ();
```

```
Customer c2=new Customer ();
```

```
Customer c3=new Customer ();
```

**OOPS1.java**

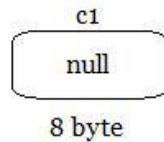
```
class OOPS1{  
    public static void main(String args[]){  
        Customer c1=new Customer();  
        c1.show();  
        Customer c2=new Customer();  
        c2.show();  
        c2.cid=99;  
        c2.cname="India";  
        c2.phone=65799999;  
        c2.show();  
    }  
}
```

```
class Customer{  
    void show() {  
        System.out.println(cid);  
        System.out.println(cname);  
        System.out.println(phone);  
    }  
}
```

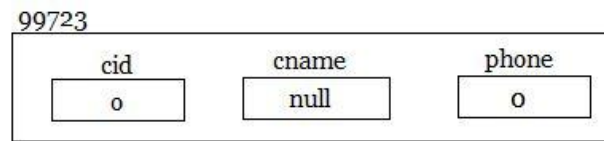
Q. What will happen inside the JVM when you create an object?

Customer c1=new Customer ();

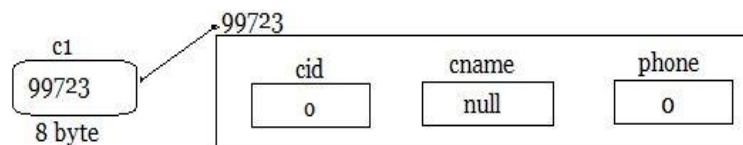
1. Allocates b bytes of memory for the interface variable and initialize with null value.



2. Verifies whether class is loaded or not. If not loaded then it loads and initializes class.
3. Creates an object i.e. Allocates the memory for instance variables of the class and initializes with the default values.



4. Address of newly created object will be assigned to the reference variable.



CLASS

- ❖ A class is an expanded concept of a structure i.e. instead of holding only data; class can hold both data and functions.
- ❖ Class contains logical description of real world entity in terms of State and Behaviour.
Class=State + Behaviour
- ❖ Class is a prototype, template, Blue print and pattern of object's state and Behaviour.
- ❖ Class is loaded only one time.
- ❖ No memory is allocated when class is created.

Syntax:

```
[modifiers] class < ClassName >
{
//Member of class
}
```

```
class Customer{
int cid;
String cname;
Long phone;
Void show () {
.....
}}
```

Example: Cars

Consider we have a Class of cars under which Santro , Alto, Maruti represent individual Objects. In this context each Car Object its own Model, Year of Manufacture, colour etc. which form Properties of the Car class and the associated action i.e. Object function like Start, Move, Stop from the Methods of Car Class.

MEMBER OF A CLASS

There are following members can be defined in class: -

1. Variables
2. Blocks
3. Constructors
4. Methods
5. Inner class

1. Variables

- ❖ Variable is the container which holds user data.
- ❖ Memory will be allocated for the variable while executing the program.
- ❖ Value of the variable can be changed an number of time during the program execution.

Type of variable based on data type: -

1. Primitive type
2. Reference type

Type of variable based on declaration: -

1. Instance variable
2. Static variable
3. Local variable

1. Instance variable

- ❖ Variable define in the class without using static modifier are called as instance variables.
- ❖ Instance variable are called as non-static variable.
- ❖ Memory will be allocated for instance variables at the time of creating an object.
- ❖ Instance variable can be primitive or reference.

```
class Hello {  
    int a;           //Primitive Instance Variable  
    String str;      //Reference Instance Variable  
}
```

- ❖ Multiple copies of memory will be allocated for multiple objects.
- ❖ Instance variable is related to instance or object of the class so when you change data of one object then that will not affect data of other object.
- ❖ Instance variable is must be accessed by reference variable which contains object of class.
- ❖ When you try to access instance variable with null reference then java.lang.NullPointerException will be thrown at run time.

Example: Hello h=null;
 h.a=90

- ❖ Following are the ways to access instance variable.
 - a. Using Reference variable that contains object reference
 Example: Hello h=new Hello ();
 h.a=90;
 - b. Directly using Object reference.
 Example: new Hello ().a=90 // (Not recommendable because no reusability)
- ❖ When you try to access instance variable with Class name then error message will be given at compile time called “Non-static variable a cannot be referenced from a static context”.
 Example: Hello. a=90;

Programs:**OOPS2.java**

```
class OOPS2{
public static void main(String args[]){
    Hello h=new Hello();
    System.out.println(h.a);
    h.a=99;
    System.out.println(h.a);
}
}
class Hello{
    int a;       //instance variable
}
```

OOPS3.java

```
class OOPS3{
public static void main(String args[]){

    Hello h1=new Hello();
    Hello h2=new Hello();
    System.out.println(h1.a+"\t"+h2.a);
    h1.a=99;
    System.out.println(h1.a+"\t"+h2.a);
}
}
class Hello{
    int a;
}
```

OOPS4.java

```
class OOPS4{
public static void main(String args[]){
new Hello().a=99;
System.out.println(new Hello().a);
}
}
class Hello{
int a;
}
```

OOPS5.java

```
class OOPS5{
public static void main(String args[]){

Hello h=null;
System.out.println(h.a);
}}
class Hello{
    int a;
}
```

OOPS6.java

```
class OOPS6{
public static void main(String args[]){
System.out.println(Hello. a);
}
}
class Hello{
int a;
}
```

OOPS7.java

```
class OOPS7{
int a;
public static void main(String args[]){

System.out.println(a);
}
}
```

Program output /error and details:-

Prog No.	output	Error and details
OOPS2	0 99	Here, 1 st load the class then create the object and allocate the memory for Instance variable with default value and again value in initialize with new value.
OOPS3	0 0 99 0	Here, Instance variable is related to instance or object of the class so when you change data of one object then that will not affect data of other object.
OOPS4	0	Here, [new Hello ().a=99;] is not reusable so it print default value.
OOPS5	Runtime error	[java.lang.NullPointerException] Because, When you try to access instance variable with null reference then NullPointerException will be thrown at run time.
OOPS6	Compilation error	[Error: non-static variable 'a' cannot be referenced from a static context]. Because, you must have to create object.
OOPS7	Compilation error	[Error: non-static variable 'a' cannot be referenced from a static context].Because, Instance variable cannot call directly.

2. Static variable

- ❖ Variable define in the class using static modifier are called as static variables.
- ❖ Static variable are called as class variable.
- ❖ Memory will be allocated for static variables at the time of loading the class.
- ❖ Static variable can be primitive or reference.

```
class Hello {
    static int b;           //Primitive static variable
    static String msg;      //Reference static variable
}
```

- ❖ Only one copy of memory will be allocated for static variable for multiple objects and single copy of memory will be accessed by all the objects.
- ❖ Static variable is related to class so when change data of static variable then modified data will be accessed by all object.
- ❖ Static variable can be accessed with class name directly and object is not required but you can access static variable with object also.
- ❖ Following are the ways to access static variable.
 - a. With class name
Example: Hello .b=90;
 - b. Using reference variable which contains null.
Example: Hello h=null;
 h. b=90;
 - c. Using the reference variable which contains object address.
Example: Hello h=new hello();
 h. b=90;

d. Directly using Object reference

Example: new Hello ().b=90;

❖ When you refer static variable with other then class name then compiler will change it with class name.

a. Hello h=null;

h. b=90; //Changes to Hello. b=90;

b. Hello h=new Hello();

h. b=90; //Changes to Hello. b=90;

c. New Hello ().b=90; //Changes to Hello. b=90;

Programs:

OOPS8.java

```
class OOPS8{
    public static void main(String args[]){
        Hello h=new Hello();
        System.out.println(h.a);
        h.a=99;
        System.out.println(h.a);
    }
    class Hello {
        static int a;
    }
}
```

OOPS9.java

```
class OOPS9{
    public static void main(String args[]){
        Hello h1=new Hello();
        Hello h2=new Hello();
        System.out.println(h1.a+"\t"+h2.a);
        h1.a=99;
        System.out.println(h1.a+"\t"+h2.a);
    }
    class Hello{
        static int a;
    }
}
```

OOPS10.java

```
class OOPS10{
    public static void main(String args[]){
        System.out.println(Hello.a);
    }
    class Hello{
        static int a;
    }
}
```

OOPS11.java

```
class OOPS11{
    public static void main(String args[]){
        Hello h=null;
        System.out.println(h.a);
    }
    class Hello{
        static int a;
    }
}
```

OOPS12.java

```
class OOPS12{
    public static void main(String args[]){
        new Hello().b=99;
        System.out.println(new Hello().b);
    }
    class Hello{
        static int b;
    }
}
```

OOPS13.java

```
class OOPS13{
    static int a;
    public static void main(String args[]){
        {
            System.out.println(a);
        }
    }
}
```

Program output /error and details:-

Prog No.	output	error and details
OOPS8	0 99	Here, 1 st load the class then allocate the memory for static variable with default value then object created and again value in initialize with new value.
OOPS9	0 0 99 99	Here, Only one copy of memory will be allocated for static variable for multiple objects and single copy of memory will be accessed by all the objects.
OOPS10	0	Here, Static variable can be accessed with class name directly.
OOPS11	0	Here, Static variable can be accessed using reference variable which contains null.
OOPS12	99	Here, Static variable can be accessed Directly using Object reference
OOPS13	0	Static variable can be accessed Directly.

3. Local variable

- ❖ Variables can be declared at two places
 - ⇒ Inside the class directly as a member
 - ⇒ Inside the members of the class
- ❖ Variables declared inside the class members like methods, Constructors and blocks are called as Local variable.
- ❖ JVM will not initialize local variable with default value; it must be initialized by you before using.
- ❖ Scope of local variable is within the class member it is declared.
- ❖ Local variable is not a member of class so it cannot be accessed using class name or object.
- ❖ Local variables cannot be static.
- ❖ Only final is allowed for local variable.

Programs:**OOPS14.java**

```

class Java544{
    public static void main(String args[]){
        Hello h=new Hello();
    }
    class Hello{
        int a=10;           //IV
        static int b=20;    //SV
        {
            int c=30;      //LV
            System.out.println(a);
            System.out.println(b);
            System.out.println(c);
        }
    }
}

```

OOPS15.java

```

class Java545{
    public static void main(String args[]){
        Hello h=new Hello();
    }
    class Hello{
        int a;
        static int b;
        {
            int c=30;
            System.out.println(a);
            System.out.println(b);
            System.out.println(c);
        }
    }
}

```

OOPS16.java

```

class OOPS16{
    public static void main(String args[]){
        Hello h=new Hello();
    }
}
class Hello{
    int a;
    static int b;
    {
        int c;
        System.out.println(a);
        System.out.println(b);
        System.out.println(c);
    }
}

```

OOPS17.java

```

class OOPS17{
    public static void main(String args[]){
    }
}
class Hello {
    static int a;
    static {
        static int b;
        System.out.println(a);
        System.out.println(b);
    }
}

```

OOPS18.java

```

class OOPS18{
    public static void main(String args[]){
        System.out.println("Main :"+Hello.a);
    }
}
class Hello {
    static int a;
    static {
        static int b=20;
        System.out.println(a);
        System.out.println(b);
    }
}

```

OOPS19.java

```

class OOPS19{
    public static void main(String args[]){
        Hello h=new Hello();
        System.out.println("main: "+h.a);
    }
}
class Hello {
    int a=10;
    {
        System.out.println("instance block 1: "+a);
    }
    {
        System.out.println("instance block 2: "+a);
    }
}

```

OOPS20.java

```

class OOPS20{
    public static void main(String args[]){
        System.out.println ("main: "+Hello. a);
    }
}
class Hello {
    static int a=10;
    static{
        System.out.println("static block 1: "+a);
    }
    static{
        System.out.println("static block 2: "+a);
    }
}

```

OOPS21.java

```

class OOPS21{
    public static void main(String args[]){
        Hello h=new Hello();
    }
}
class Hello {
    int a=10;
    {
        System.out.println("Ins block 1 : "+a);
    }
    {
        String str="INDIA";
        System.out.println("Ins block 2 : "+str);
    }
}

```


OOPS22.java

```
class OOPS22{
    public static void main(String args[]){
        Hello h=new Hello();
    }
    class Hello    {
        {
            int a=10;
            System.out.println("Ins block 1 : "+a);
        }
        {
            System.out.println("Ins block 2 : 
"+str);
        }
    }
}
```

OOPS23.java

```
class OOPS23{
    public static void main(String args[]){
        Hello h=new Hello();
        System.out.println("Main : "+h.a);
    }
    class Hello    {
        {
            int a=10;
            System.out.println("Ins block: "+a);
        }
    }
}
```

OOPS24.java

```
class OOPS24{
    public static void main(String args[]){
        System.out.println("main: "+Hello.a);
    }
}
class Hello    {
    static{
        int a=10;
        System.out.println("s block: "+a);
    }
}
```

Program output /error and details:-

Prog No.	output	error and details
OOPS14	10 20 30	Here, 'a' is Instance variable and b is Static variable and c is local variable.
OOPS15	0 0 30	Here, JVM will initialize Instance variable and Static variable with default value.
OOPS16	Compilation error	[Error: variable c might not have been initialized]. JVM will not initialize local variable with default value.
OOPS17	Compilation error	[Error: illegal start of expression (static int b;), Local variables cannot be static.
OOPS18	0 20 Main: 0	Here, final is allowed for local variable.
OOPS19	instance block: 10 instance block: 10 main: 10	Here, class loaded then Object created so that Instance block will be executed.
OOPS20	static block: 10 static block: 10 main: 10	Here, class loaded then static block will executed.
OOPS21	Ins block 1: 10 Ins block 2: INDIA	Scope of variable is within the block so first time value of a is 10 and 2 nd time is INDIA
OOPS22	Compilation error	[Cannot find symbol a] because, 2 nd instance block is not able to access to a value.
OOPS23	Compilation error	[Cannot find symbol a] because, variable 'a' is define as local variable it cannot access outside.
OOPS24	Compilation error	[Cannot find symbol a] because, variable 'a' is define as local variable it cannot access outside.